

GOOGLE CYBERSECURITY CERTIFICATE

Cryptography

Google Cybersecurity Professional Certificate · Study Guide

Two hands-on Linux cryptography labs: verifying file integrity with SHA-256 hashing, and decrypting a message protected first by a Caesar cipher and then by AES-256-CBC.

Track	Google Cybersecurity Professional Certificate
Course	Assets, Threats, and Vulnerabilities
Environment	Linux Bash shell (/home/analyst)
Analyst	Chayan Panchal

1 Lab 1 — Generating and Comparing File Hashes

Two files, file1.txt and file2.txt, look byte-for-byte identical when printed with cat — both contain the EICAR antivirus test string. The task is to prove, using hashing, whether they're actually the same file.



Both files appear identical on screen

```
$ cat file1.txt
X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
$ cat file2.txt
X50!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```



sha256sum reveals the files are NOT identical

```
$ sha256sum file1.txt
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdf8267  file1.txt
$ sha256sum file2.txt
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
```

WHY THIS MATTERS Visual inspection said the files matched; the hash proved otherwise. This is exactly why integrity checks in security work rely on cryptographic hashes rather than eyeballing content — a single differing byte anywhere in the file produces a completely different digest.



cmp confirms the divergence at the first byte

```
$ sha256sum file1.txt >> file1hash
$ sha256sum file2.txt >> file2hash
$ cmp file1hash file2hash
file1hash file2hash differ: char 1, line 1
```

2 Lab 2 — Caesar Cipher + AES Decryption Chain

A home directory contains an encrypted file (Q1.encrypted), a README pointing to a hidden subdirectory, and inside that subdirectory a hidden file encrypted with a classic Caesar cipher — a three-stage puzzle that mirrors how a real ransomware note or CTF-style challenge might be laid out.

Decryption Chain

1. FIND THE HIDDEN FILE

```
ls -a inside caesar/ reveals .leftShift3
```

2. SOLVE THE CAESAR CIPHER

```
tr shifts each letter back 3 positions
```

3. DECRYPT WITH THE REVEALED KEY

```
openssl aes-256-cbc recovers Q1.encrypted
```

Decrypting the Caesar cipher with tr (left-shift-3)

```
$ cat .leftShift3
Lq rughu wr uhfryhu brxu ilohv brx zloo qhng wr hqwhu wkh iroorzlqj frppdgg:
rshqvvo dhv-256-fef -sengi2 -d -g -lq T1.hqfubswgh -rxw T1.uhfryhuhg -n hwwxeuxwh

$ cat .leftShift3 | tr "d-za-cD-ZA-C" "a-zA-Z"
In order to recover your files you will need to enter the following command:
openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
```

NOTE The tr command maps one character set onto another position-for-position. "d-za-cD-ZA-C" -> "a-zA-Z" shifts every letter three places back in the alphabet, which is exactly what's needed to undo a left-shift-3 Caesar cipher.

Final AES-256-CBC decryption using the key recovered from the cipher

```
$ openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted -out Q1.recovered -k ettubrute
$ cat Q1.recovered
If you are able to read this, then you have successfully decrypted the classic
cipher text. You recovered the encryption key that was used to encrypt this file.
```

3 Quick Reference



Cryptography — quick reference

```
sha256sum <file>                # generate a SHA-256 hash
cmp file1hash file2hash          # byte-by-byte diff between two files
cat file | tr "d-za-cD-ZA-C" "a-zA-Z"  # undo a left-shift-3 Caesar cipher
openssl aes-256-cbc -pbkdf2 -a -d -in IN -out OUT -k KEY  # AES-256-CBC decrypt
```